Problem 8

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
# from sklearn.datasets import load_breast_cancer
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score,
confusion_matrix

iris = load_iris()
X = iris.data
y = iris.target_names[iris.target]

# bc = load_breast_cancer()
# X = bc.data
# y = bc.target_names[bc.target]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

clf = DecisionTreeClassifier()
clf = clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)

disp = ConfusionMatrixDisplay(cm, display_labels=clf.classes_)
disp.plot()
plt.show()
plot_tree(clf, filled=True)
```

Problem 9

```
mport random as rd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_olivetti_faces
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

face = fetch_olivetti_faces()
plt.imshow(face.images[rd.randint(0,400)], cmap='grey')
plt.show()
```

```
X = face.data
y = face.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

clf = GaussianNB()
clf = clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
```

problem 10

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_breast_cancer
# from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# X, y = load_iris(return_X_y=True)

X, y = load_breast_cancer(return_X_y=True)

print(f'First 5 rows of the given data:\n {X[:5]}')
X_sc = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pc = pca.fit_transform(X_sc)
print(f'First 5 rows of the reduced data:\n {X_pc[:5]}')

n = 2
km = KMeans(n_clusters=n).fit(X_pc)
centroids = km.cluster_centers_
y_km= km.predict(X_pc)

y_km = y_km.reshape(-1,1)
data = np.append(X_pc, y_km, axis=1)
print(f'First 5 rows of the Classified data:\n {data[:5]}')


plt.scatter(X_pc[:, 0], X_pc[:, 1], c=y_km, cmap='viridis')
```

```
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=100,
alpha=0.5, label='Centroids')
plt.title('K-means Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()

# plt.scatter(X_pc[:, 0], X_pc[:, 1], c=y, cmap='viridis')
# plt.show()
```